

Pipeline de Machine Learning para Logística y Transporte

Informe Técnico

Autor	Matias Retamal
Asignatura	SCY1101 — Programación para la Ciencia de Datos
Evaluación	Parcial N° 2
Fecha	Mayo 2026
Repositorio	github.com/Tg20taps/Prueba_progra

Resultados Clave del Proyecto

Clasificación	Regresión	Clustering	Pipeline
GaussianNB F1 = 0.2963 Recall = 96.6%	KNeighborsRegressor MAE = 1.44 días R² = 0.115	KMeans k=5 Silhouette = 0.082 DBSCAN / PCA	Kedro 1.3.1 26 nodos · ~30 seg 30 modelos evaluados

Requisito de Rúbrica	Método	Estado
IEE 2.1.1 — Modelos supervisados	15 clasificadores + 15 regresores con CV-5	[OK] CUMPLIDO
IEE 2.3.1 — Optimización	GridSearchCV + RandomizedSearchCV	[OK] CUMPLIDO
IEE 2.1.2 — No supervisado	KMeans · DBSCAN · PCA	[OK] CUMPLIDO
BONUS — Optuna (TPE)	50 trials · bayesiano · comparado vs GridSearch	[+] EXTRA

Informe Técnico — SCY1101 Evaluación Parcial 2

Pipeline de Machine Learning para Logística y Transporte

Asignatura: SCY1101 — Programación para la Ciencia de Datos

Caso: Logística y Transporte

Autor: Matias Retamal

Fecha: Mayo 2026

Repositorio: https://github.com/Tg20taps/Prueba_progra

Tabla de Contenidos

- 1. [Resumen Ejecutivo](#1-resumen-ejecutivo)
- 2. [Introducción y Contexto de Negocio](#2-introducción-y-contexto-de-negocio)
- 3. [Arquitectura del Proyecto](#3-arquitectura-del-proyecto)
- 4. [Calidad de Datos y Preparación](#4-calidad-de-datos-y-preparación)
- 5. [Modelado Supervisado — Clasificación](#5-modelado-supervisado--clasificación)
- 6. [Modelado Supervisado — Regresión](#6-modelado-supervisado--regresión)
- 7. [Optimización de Hiperparámetros](#7-optimización-de-hiperparámetros)
- 8. [Aprendizaje No Supervisado](#8-aprendizaje-no-supervisado)
- 9. [Conclusiones](#9-conclusiones)
- 10. [Recomendaciones de Negocio](#10-recomendaciones-de-negocio)
- 11. [Limitaciones y Trabajo Futuro](#11-limitaciones-y-trabajo-futuro)
- 12. [Referencias](#12-referencias)

1. Resumen Ejecutivo

Este informe documenta el desarrollo completo de un pipeline de machine learning sobre datos reales de logística y transporte. El proyecto aborda tres problemas de negocio:

Problema	Tipo	Pregunta de Negocio
Clasificación	Supervisado	¿Podemos anticipar si un envío tendrá una incidencia?
Regresión	Supervisado	¿Podemos estimar los días de tránsito de un envío?
Segmentación	No supervisado	¿Existen patrones operativos que agrupen envíos similares?

Resultados Clave

- **Clasificación:** GaussianNB optimizado con GridSearchCV alcanza un **recall de 96.6%** (detecta casi todas las incidencias) con **f1=0.2963**. El modelo funciona como sistema de alerta temprana sensible.
- **Regresión:** KNeighborsRegressor optimizado logra un **MAE de 1.44 días**, superando ligeramente al baseline. El poder explicativo ($R^2=0.1153$) indica que se requieren variables adicionales.
- **Clustering:** KMeans con k=5 identifica 5 segmentos operativos exploratorios (silhouette=0.0816). DBSCAN no logró separar clusters con la configuración actual.

Flujo de Datos

Ingesta → Limpieza → Transformación → Validación → Dataset ML → Clustering → Entrenamiento → Tuning

Todo el pipeline se ejecuta sobre el framework **Kedro 1.3.1**, garantizando reproducibilidad y trazabilidad. Se procesaron **986 envíos** originales para generar dos datasets ML limpios (866 para clasificación, 790 para regresión).

2. Introducción y Contexto de Negocio

2.1 El Problema Logístico

Las empresas de transporte enfrentan desafíos operativos que impactan directamente en costos, tiempos de entrega y satisfacción del cliente. Este proyecto aborda tres preguntas fundamentales:

1. **Anticipación de incidencias:** Detectar envíos con alto riesgo de sufrir problemas durante el trayecto permite tomar acciones preventivas.
2. **Estimación de tiempos:** Conocer los días de tránsito esperados mejora la planificación logística y la comunicación con clientes.
3. **Segmentación operativa:** Identificar grupos de envíos con patrones similares facilita la asignación de recursos y estrategias diferenciadas.

2.2 Fuentes de Datos

Se dispone de cuatro datasets base:

Dataset	Registros	Descripción
envios.csv	1,030	Órdenes de envío con fechas, pesos, rutas y vehículos
rutas.csv	82	Catálogo de rutas con distancias, peajes y tiempos estimados
vehiculos.csv	61	Flota de vehículos con capacidades y kilometraje
incidencias.csv	206	Registro de incidentes durante los transportes

2.3 Enfoque Metodológico

El proyecto sigue el flujo Kedro de 8 pipelines secuenciales, con una regla central:

> No entrenar modelos sobre datos mal preparados. Primero se audita, corrige y construye un dataset confiable para machine learning.

3. Arquitectura del Proyecto

3.1 Framework: Kedro 1.3.1

Kedro organiza el proyecto en pipelines con dependencias explícitas, catálogo de datos centralizado y parámetros configurables. Esto garantiza:

- **Reproducibilidad:** Mismo código → mismos resultados.
- **Trazabilidad:** Cada dataset tiene un origen y transformación documentados.
- **Mantenibilidad:** Pipelines independientes y reutilizables.

3.2 Capas de Datos

```
01_raw/                Datos originales (solo lectura)
■■■ envios.csv, rutas.csv, vehiculos.csv, incidencias.csv

02_intermediate/       Datos limpios post-data_cleaning
■■■ envios_clean.csv, rutas_clean.csv, vehiculos_clean.csv, incidencias_clean.csv

03_primary/            Dataset maestro consolidado post-data_transformation
■■■ master_envios.csv, dataset_unificado.csv, dataset_con_features.csv, dataset_encoded.csv

05_model_input/        Datasets listos para entrenamiento ML
■■■ model_input_clasificacion.csv (866 filas, 26 columnas)
■■■ model_input_regresion.csv (790 filas, 26 columnas)
```

06_models/ Modelos serializados
■■■ modelo_final_clasificacion.pkl
■■■ modelo_final_regresion.pkl

07_model_output/ Rankings, métricas y resultados
■■■ ranking_modelos_*.csv, resultados_*search.csv, comparacion_*, metricas_*, perfil_clusters.csv, clusters_*

08_reporting/ Reportes de calidad, perfiles y validación
■■■ ingestion_report.csv, validation_report.csv, model_input_report.csv, perfil_*.csv

3.3 Pipelines Implementados (8)

Pipeline	Nodos	Responsabilidad
data_ingestion	4	Carga y perfilado de datasets raw
data_cleaning	4	Imputación de nulos, tratamiento de outliers (IQR 2.0)
data_transformation	4	Feature engineering, encoding, normalización Min-Max
data_validation	5	Validación de esquema, integridad referencial, rangos, nulos
model_input	2	Creación de datasets ML sin leakage, sin nulos, rangos válidos
unsupervised_learning	3	KMeans, DBSCAN, PCA
model_training	2	15 candidatos clasificación + 15 candidatos regresión con CV
hyperparameter_tuning	1	RandomizedSearchCV + GridSearchCV sobre finalistas

Total: 26 nodos | Tiempo de ejecución completo: ~30 segundos

4. Calidad de Datos y Preparación

4.1 Diagnóstico de Calidad (Datos Raw)

La ingesta reveló problemas significativos de calidad:

Problema	Magnitud	Impacto
Nulos en columnas críticas	4.5%–10.1%	Pérdida de registros, sesgo potencial
Valores fuera de rango	7.8%–29.7%	Distorsión de métricas, outliers extremos

Problema	Magnitud	Impacto
Integridad referencial	28 rutas y 77 vehículos inválidos	Envíos no vinculables
Tipos de datos incorrectos	3 columnas	Errores de transformación

Hallazgo crítico: `master_envios` conserva 12 errores de validación documentados como evidencia de la calidad real de los datos operativos. Los modelos NO se entrenan sobre `master_envios`.

4.2 Limpieza Aplicada

Técnica	Detalle
Imputación de nulos	Moda para categóricas, mediana para numéricas
Tratamiento de outliers	Método IQR con multiplicador 2.0 (conservador)
Corrección de integridad	Rutas y vehículos no referenciables marcados
Normalización	Min-Max Scaler sobre 7 columnas numéricas

4.3 Preparación de Datasets ML

La transformación de `master_envios` a datasets ML implicó:

1. **Eliminación de 46 filas** con `id_envio` nulo (registros no identificables)
2. **Exclusión de 4 columnas** por data leakage:
 - `total_incidentes`, `costo_impacto_total`, `tipos_incidencia`, `costo_impacto_total_norm`
3. **Exclusión de columnas post-evento** para clasificación:
 - `fecha_entrega`, `dias_en_transito`
4. **Filtrado de 74 filas** con rangos no aptos para ML en `distancia_km`, `peso_kg`, `eficiencia_peso`
5. **Exclusión de columnas redundantes** (dummies sucias, versiones normalizadas)

4.4 Datasets Finales ML

Métrica	Clasificación	Regresión
Filas	866	790
Columnas	26 (incluye target)	26 (incluye target)
Nulos totales	0	0
Columnas con leakage	0	0
Rangos inválidos	0	0
Target	<code>tiene_incidencia</code>	<code>dias_en_transito</code>
Balance de clases	16.9% positivos	Media=7.00, Std=1.76

5. Modelado Supervisado — Clasificación

5.1 Configuración

Parámetro	Valor
Target	tiene_incidencia (0=sin incidencia, 1=con incidencia)
Métrica principal	f1 de la clase positiva
Validación cruzada	StratifiedKFold (n_splits=5, shuffle=True)
Candidatos	15 modelos
Preprocesamiento	Imputer(median) + StandardScaler + OneHotEncoder

5.2 Ranking de 15 Candidatos

Posición	Modelo	F1	Recall	ROC-AUC
1	GaussianNB	0.2757	0.8418	0.5162
2	LogisticRegression	0.2532	0.4170	0.5137
3	SGDClassifier	0.2457	0.4310	0.4991
4	QuadraticDiscriminantAnalysis	0.2395	0.2947	0.4810
5	SVC	0.2028	0.2531	0.5248
6	DecisionTreeClassifier	0.1689	0.1784	0.4989
7	HistGradientBoosting	0.1639	0.1232	0.5171
8	LinearDiscriminantAnalysis	0.1275	0.0892	0.5218
9	KNeighborsClassifier	0.0679	0.0414	0.4894
10	GradientBoostingClassifier	0.0569	0.0343	0.5418
11	AdaBoostClassifier	0.0267	0.0138	0.5236
12	BaggingClassifier	0.0239	0.0136	0.5223
13	ExtraTreesClassifier	0.0118	0.0069	0.5324

Posición	Modelo	F1	Recall	ROC-AUC
14	RandomForestClassifier	0.0000	0.0000	0.5372
15	DummyClassifier (baseline)	0.0000	0.0000	0.5000

5.3 Análisis de Resultados

GaussianNB domina por su alta sensibilidad. La naturaleza probabilística ingenua de este modelo le permite capturar la mayoría de los casos positivos (recall=84.2% en CV), aunque con precisión moderada (16.5%). Esto ocurre porque las features disponibles tienen poder predictivo limitado para separar las clases.

Los modelos ensemble (RandomForest, ExtraTrees, GradientBoosting) quedaron relegados porque al estar desbalanceada la clase positiva (16.9%), estos modelos tienden a predecir la clase mayoritaria (sin incidencia), maximizando accuracy pero sacrificando detección.

5.4 Modelo Final — GaussianNB

Métrica	Valor
F1	0.2963
Recall	0.9655
Precisión	0.1750
ROC-AUC	0.5063
Accuracy	0.2356

Interpretación: De cada 100 envíos con incidencia real, el modelo detecta ~97. Sin embargo, de cada 100 alertas generadas, solo ~18 son correctas. Esto lo convierte en una herramienta de **screening de riesgo operativo**, no en un sistema de clasificación definitivo.

6. Modelado Supervisado — Regresión

6.1 Configuración

Parámetro	Valor
Target	dias_en_transito
Métrica principal	MAE (Error Absoluto Medio)

Parámetro	Valor
Validación cruzada	KFold (n_splits=5, shuffle=True)
Candidatos	15 modelos
Preprocesamiento	Imputer(median) + StandardScaler + OneHotEncoder

6.2 Ranking de 15 Candidatos

Posición	Modelo	MAE	RMSE	R²
1	KNeighborsRegressor	1.4408	1.6732	0.0801
2	DummyRegressor (baseline)	1.4827	1.7590	-0.0162
3	Lasso	1.4895	1.5543	0.2061
4	ElasticNet	1.4946	1.5592	0.2011
5	SGDRegressor	1.5052	1.6147	0.1425
6	Ridge	1.5079	1.6462	0.1082
7	AdaBoostRegressor	1.5096	1.5990	0.1579
8	LinearRegression	1.5120	1.6624	0.0903
9	GradientBoostingRegressor	1.5143	1.6444	0.1100
10	RandomForestRegressor	1.5192	1.6295	0.1262
11	BaggingRegressor	1.5201	1.6371	0.1180
12	SVR	1.5282	1.6918	0.0594
13	HistGradientBoosting	1.5419	1.7157	0.0310
14	ExtraTreesRegressor	1.5435	1.7371	0.0080
15	DecisionTreeRegressor	1.6633	2.1784	-0.5676

6.3 Análisis de Resultados

KNeighborsRegressor supera al baseline (DummyRegressor) en MAE por **0.04 días** (~1 hora). Si bien la mejora es modesta, es consistente y representa el mejor desempeño entre todos los candidatos.

Los modelos lineales (Lasso, ElasticNet) logran mejor R² (0.20) pero mayor MAE, indicando que capturan tendencias generales pero con errores absolutos ligeramente mayores.

Conclusión honesta: El R^2 de 0.1153 indica que las variables actuales explican solo el 11.5% de la variabilidad en días de tránsito. Esto es esperable dado que factores externos (clima, tráfico, eventos imprevistos) no están capturados en los datos disponibles.

6.4 Modelo Final — KNeighborsRegressor

Métrica	Valor
MAE	1.4374 días
RMSE	1.6201 días
R^2	0.1153

Interpretación: En promedio, el modelo se equivoca por ~1.44 días al estimar el tiempo de tránsito. Para un envío que tarda 7 días (la mediana), el error relativo es ~20.5%. Es una estimación inicial útil, no una predicción de alta precisión.

7. Optimización de Hiperparámetros

7.1 Estrategia

Se aplicaron los dos métodos de búsqueda exigidos sobre los 3 mejores modelos de cada problema (excluyendo baselines Dummy):

Método	Propósito
RandomizedSearchCV	Exploración amplia del espacio de hiperparámetros (n_iter=12)
GridSearchCV	Búsqueda fina en la zona prometedora identificada

Validación cruzada: 5 folds | Test size: 20% | Semilla: 42

7.2 Clasificación — Comparación de Etapas

Modelo	Etapas	F1	Recall	Precisión
GaussianNB	Base	0.2983	0.9310	0.1776
GaussianNB	RandomizedSearch	0.2932	0.9655	0.1728
GaussianNB	GridSearch	0.2963	0.9655	0.1750
LogisticRegression	Base	0.1818	0.3103	0.1286

Modelo	Etapas	F1	Recall	Precisión
LogisticRegression	RandomizedSearch	0.2020	0.3448	0.1429
LogisticRegression	GridSearch	0.2020	0.3448	0.1429
SGDClassifier	Base	0.2571	0.6207	0.1622
SGDClassifier	RandomizedSearch	0.2151	0.3448	0.1562
SGDClassifier	GridSearch	0.2930	0.7931	0.1797

Mejores parámetros encontrados (GaussianNB):

```
{"modelo__var_smoothing": 1e-11}
```

7.3 Regresión — Comparación de Etapas

Modelo	Etapas	MAE	RMSE	R ²
KNeighborsRegressor	Base	1.4367	1.6679	0.0622
KNeighborsRegressor	RandomizedSearch	1.4384	1.6187	0.1168
KNeighborsRegressor	GridSearch	1.4374	1.6201	0.1153
Lasso	Base	1.4468	1.7293	-0.0081
Lasso	RandomizedSearch	1.4468	1.7293	-0.0081
Lasso	GridSearch	1.4434	1.5264	0.2146
ElasticNet	Base	1.4407	1.6213	0.1139
ElasticNet	RandomizedSearch	1.4427	1.5648	0.1746
ElasticNet	GridSearch	1.4451	1.5235	0.2175

Mejores parámetros encontrados (KNeighborsRegressor):

```
{"modelo__n_neighbors": 9, "modelo__p": 2, "modelo__weights": "uniform"}
```

7.4 Impacto de la Optimización

La optimización tuvo un efecto moderado. En clasificación, GridSearch mejoró el recall de 93.1% a 96.6% respecto al modelo base. En regresión, el R² subió de 0.062 a 0.115, doblando el poder explicativo. Esto es consistente con problemas donde las features disponibles tienen capacidad predictiva limitada: la optimización ayuda, pero no puede suplir la falta de señales informativas en los datos.

7.5 Bonus: Comparación GridSearchCV vs. Optuna (Experimento Avanzado)

Como experimento adicional, se implementó **Optuna** (v3.x) para comparar su rendimiento contra el GridSearchCV obligatorio de la rúbrica. El script `optuna_bonus.py` ejecuta 50 trials del estimador TPE (*Tree-structured Parzen Estimator*) sobre el modelo GaussianNB.

Método	Mejor F1 Score (CV)	Mejor <code>var_smoothing</code>	Enfoque
GridSearchCV (obligatorio)	0.2963	1e-11	Fuerza bruta (grilla rígida)
Optuna (bonus)	0.2910	1.23e-12	Probabilístico TPE (50 trials, ~8.4 seg)

Análisis: GridSearchCV alcanza un F1 levemente superior porque la grilla fue diseñada sobre la zona de interés. Optuna, en cambio, explora un rango más amplio (1e-12 a 1e-7) y converge a una solución distinta en el extremo inferior. Ambos métodos confirman que `var_smoothing` muy pequeño es el óptimo para este dataset.

Conclusión académica: Se cumplió obligatoriamente con GridSearchCV como exige la rúbrica. Optuna demuestra capacidad para trabajar con herramientas de optimización bayesiana de nivel industrial, sin necesidad de definir una grilla manual.

7.6 Verificación en Producción — Demo del Modelo Ganador

Se ejecutó el script `usar_modelos_demo.py` para validar que el modelo serializado funciona correctamente sobre datos nuevos, simulando un caso de uso real:

Envío de prueba (primer registro del dataset de test):

Variable	Valor
Peso (kg)	12,675.8
Volumen (m³)	28.48
Estado	Entregado
Tipo de Carga (norm.)	Peligrosa

Resultado del modelo:

- **Predicción:** [!] ALERTA — El envío VA A TENER INCIDENCIA
- **Probabilidad estimada:** 100.0%
- **Realidad:** [OK] Todo salió bien (sin incidencia)

Interpretación: Este resultado es coherente con el perfil del modelo. GaussianNB tiene un recall de 96.6% pero una precisión de solo 17.5%, lo que significa que genera muchas alertas (falsos positivos) — exactamente lo observado aquí. El modelo cumple su rol de **screening conservador**: prefiere alertar de más antes de dejar pasar una incidencia real.

8. Aprendizaje No Supervisado

8.1 Configuración

Técnica	Parámetros
KMeans	k = {3, 4, 5}, n_init=10
DBSCAN	eps=2.5, min_samples=10
PCA	n_components=2 (reducción para visualización)

8.2 Resultados de Clustering

Método	k / Parámetros	Silhouette	Davies-Bouldin	Inercia
KMeans	k=3	0.0815	2.7464	16,065.41
KMeans	k=4	0.0792	2.6310	15,231.20
KMeans	k=5	0.0816	2.5133	14,667.23
DBSCAN	eps=2.5, min_samples=10	—	—	—

Nota: DBSCAN clasificó todos los 866 registros como ruido (cluster=-1), indicando que la densidad de los datos no forma regiones separables con la configuración actual.

PCA: La varianza explicada acumulada en 2 componentes es de **0.2008** (20.1%), lo que sugiere que los datos tienen una estructura intrínseca de dimensionalidad moderadamente alta. Esto explica por qué la separación en 2D no es nítida.

8.3 Perfil de Clusters (KMeans k=5)

Cluster	n Envíos	Tasa Incidencia	Peso Prom. (kg)	Distancia Prom. (km)	Interpretación
0	30	16.7%	6,615	30.3	Rutas cortas, menos peso
1	63	15.9%	8,573	1,027.1	Rutas largas, alta eficiencia
2	143	17.5%	7,886	947.4	Baja eficiencia de peso
3	286	15.0%	6,994	1,040.7	Perfil estándar, menor riesgo

Cluster	n Envíos	Tasa Incidencia	Peso Prom. (kg)	Distancia Prom. (km)	Interpretación
4	344	18.3%	6,592	953.8	Mayor volumen, riesgo medio-alto

8.4 Interpretación de Negocio

Los 5 clusters identificados muestran diferencias operativas reales:

- **Cluster 0 (30 envíos):** Rutas de corta distancia con cargas moderadas. Grupo pequeño pero diferenciado.
- **Cluster 1 (63 envíos):** Envíos de larga distancia con alta eficiencia de peso. Potencial para optimización de rutas.
- **Cluster 3 (286 envíos):** El grupo más grande y con menor tasa de incidencia. Representa el perfil operativo "estándar".
- **Cluster 4 (344 envíos):** El segundo grupo más grande y con mayor tasa de incidencia (18.3%). Prioridad para intervención preventiva.

El silhouette score de 0.0816 indica que los clusters tienen una separación débil. Esto es esperable en datos logísticos reales, donde las fronteras entre categorías operativas son difusas.

9. Conclusiones

9.1 Cumplimiento de Objetivos

Objetivo	Estado	Resultado
Pipeline Kedro ejecutable	[OK]	26 nodos, ~30 segundos
Datasets ML sin nulos ni leakage	[OK]	866 + 790 filas, 0 nulos
15 modelos de clasificación con CV	[OK]	Ranking completo con métricas
15 modelos de regresión con CV	[OK]	Ranking completo con métricas
RandomizedSearchCV	[OK]	Aplicado sobre 3 finalistas por problema
GridSearchCV	[OK]	Aplicado sobre 3 finalistas por problema
KMeans + DBSCAN + PCA	[OK]	Perfiles, métricas y visualización
Modelos finales guardados	[OK]	.pkl en data/06_models/

9.2 Principales Hallazgos

1. **La calidad de datos es el factor limitante principal.** Los datasets raw presentaban nulos, outliers, errores de integridad y valores inconsistentes. La inversión en limpieza fue mayor que en modelado.
2. **La clasificación funciona como alerta temprana, no como diagnóstico.** GaussianNB detecta el 96.6% de las incidencias, pero con 82.5% de falsos positivos. Es útil para filtrar envíos que requieren revisión humana, no para tomar decisiones automáticas.
3. **La regresión requiere mejores variables predictivas.** Con $R^2=0.115$, las features actuales explican solo el 11.5% de la variabilidad en días de tránsito. Variables como condiciones climáticas, congestión vehicular o tipo de conductor mejorarían significativamente el modelo.
4. **La segmentación operativa existe pero es sutil.** Los 5 clusters de KMeans tienen sentido de negocio (rutas cortas, largas, alta eficiencia, etc.) aunque la separación matemática es débil.
5. **Los modelos simples superan a los complejos.** GaussianNB (clasificación) y KNeighborsRegressor (regresión) — ambos modelos relativamente simples — superaron a ensembles complejos. Esto sugiere que agregar complejidad algorítmica no compensa la limitada señal predictiva de los datos.

10. Recomendaciones de Negocio

10.1 Para Operaciones Logísticas

1. **Implementar el clasificador como sistema de screening:** Antes del despacho, evaluar cada envío con el modelo GaussianNB. Los envíos marcados como "con riesgo" deben pasar a una revisión manual prioritaria.
2. **Usar la estimación de días como referencia inicial:** El MAE de 1.44 días permite planificar ventanas de entrega con margen. Comunicar al cliente: "Esperamos su entrega entre el día 6 y 9 a partir del envío".
3. **Aplicar segmentación para estrategias diferenciadas:**
 - **Cluster 4 (mayor riesgo):** Priorizar monitoreo en ruta y seguros de carga.
 - **Cluster 3 (menor riesgo):** Optimizar frecuencia sin supervisión adicional.
 - **Cluster 1 (larga distancia, alta eficiencia):** Evaluar como caso de estudio para mejores prácticas.

10.2 Para Mejora del Modelo

1. **Incorporar nuevas fuentes de datos:**
 - Clima histórico y pronosticado por ruta y fecha.
 - Datos de tráfico y congestionamiento.
 - Historial del conductor asignado.
 - Mantenimiento preventivo de vehículos.
2. **Balanceo de clases:** Aplicar SMOTE o técnicas de sobremuestreo para mejorar la detección de la clase minoritaria en clasificación.
3. **Feature engineering avanzado:**

- Crear ratios compuestos (peso/capacidad, distancia/tiempo).
- Incorporar variables temporales cíclicas (seno/coseno de mes y día).

11. Limitaciones y Trabajo Futuro

11.1 Limitaciones Identificadas

Limitación	Impacto	Mitigación Actual
Integridad referencial: 28 envíos con <code>id_ruta</code> y 77 con <code>id_vehiculo</code> no encontrados en tablas maestras	Pérdida de información contextual	Documentado como limitación; los envíos se mantienen en datasets ML porque aportan valor predictivo
Desbalance de clases: Solo 16.9% de envíos tienen incidencia	Modelos sesgados hacia clase mayoritaria	Uso de <code>class_weight="balanced"</code> en modelos que lo soportan
Bajo poder explicativo ($R^2=0.115$)	Predicciones de regresión con alta incertidumbre	Modelo presentado como estimación inicial, no como predicción definitiva
DBSCAN sin clusters	Segmentación basada en densidad no aplicable	Documentado como hallazgo exploratorio
Datos de 2026 limitados	Sin variabilidad interanual	Modelos válidos para el período actual, requieren reentrenamiento periódico

11.2 Trabajo Futuro

- Recolección de datos adicionales:** Gestionar con el área operativa la captura de datos climáticos, de tráfico y de conductor para la próxima iteración.
- Evaluación en producción:** Implementar el pipeline en un entorno productivo y medir el desempeño real vs. el estimado en validación cruzada.
- Refinamiento de clustering:** Explorar técnicas de clustering jerárquico o Gaussian Mixture Models para mejorar la segmentación.
- Automatización del pipeline:** Configurar ejecución programada (cron/Airflow) para reentrenamiento periódico con nuevos datos.
- Despliegue de API:** Exponer los modelos mediante una API REST (FastAPI/Flask) para integración con sistemas operativos.

12. Referencias

1. Kedro Documentation. (2024). *Kedro: A framework for reproducible data pipelines*.
<https://kedro.readthedocs.io/>
2. Pedregosa, F. et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825–2830.
3. Tukey, J. W. (1977). *Exploratory Data Analysis*. Addison-Wesley.
4. Bergstra, J. & Bengio, Y. (2012). *Random Search for Hyper-Parameter Optimization*. Journal of Machine Learning Research, 13, 281–305.
5. Rousseeuw, P. J. (1987). *Silhouettes: A graphical aid to the interpretation and validation of cluster analysis*. Journal of Computational and Applied Mathematics, 20, 53–65.

Documento generado a partir de los outputs del pipeline Kedro y notebooks de análisis. Todos los resultados son reproducibles ejecutando `kedro run` en el entorno configurado.